

INFORME PROYECTO SO – PARTE 1

SISTEMA DE MONITOREO Y PRUEBAS DE ESTRÉS SOBRE BASE DE DATOS POSTGRESQL MEDIANTE DOCKER Y NEXT.JS

Estudiantes

Jhoan Hurtado Marin – 202459472

Juan Stevan Cruz – 202459437

Asignatura

Sistemas Operativos

Resumen

Este trabajo presenta el diseño e implementación de una plataforma de monitoreo y pruebas de estrés para una base de datos PostgreSQL desplegada mediante contenedores Docker. La solución integra una aplicación web desarrollada con Next.js capaz de visualizar métricas de CPU, memoria RAM y tiempos de respuesta en tiempo real. Para validar el comportamiento del sistema se realizaron consultas progresivas sobre una base de datos con más de 520.000 registros distribuidos en diferentes tablas. Los resultados obtenidos evidencian que PostgreSQL mantiene tiempos de respuesta aceptables incluso bajo cargas elevadas, alcanzando consultas cercanas al medio millón de registros en aproximadamente 1.17 segundos, conservando estabilidad en el consumo de recursos del sistema.

Palabras clave: PostgreSQL, Docker, Next.js, pruebas de estrés, monitoreo, rendimiento, bases de datos.

1. Introducción

Las bases de datos constituyen uno de los componentes más importantes dentro de las aplicaciones modernas, ya que almacenan y gestionan grandes volúmenes de información. A medida que aumenta la cantidad de usuarios y transacciones, resulta

necesario evaluar el comportamiento del sistema bajo diferentes condiciones de carga.

Las pruebas de estrés permiten identificar límites operativos, cuellos de botella y posibles problemas de rendimiento antes de que una aplicación entre en producción. Paralelamente, el monitoreo en tiempo real facilita la observación del consumo de recursos del sistema, proporcionando información valiosa para la toma de decisiones relacionadas con optimización y escalabilidad.

En este proyecto se desarrolló una plataforma web capaz de ejecutar consultas de alta carga sobre PostgreSQL y visualizar simultáneamente métricas relacionadas con CPU, memoria RAM y tiempos de respuesta. La solución fue implementada utilizando Docker para la virtualización de servicios, PostgreSQL como sistema gestor de bases de datos y Next.js para la construcción del dashboard de monitoreo.

2. Objetivos

2.1 Objetivo General

Desarrollar una aplicación web para realizar pruebas de estrés sobre una base de datos PostgreSQL y monitorear el comportamiento de los recursos del sistema en tiempo real.

2.2 Objetivos Específicos

- Implementar PostgreSQL utilizando contenedores Docker.
 - Construir una interfaz web para la visualización de métricas del sistema.
 - Monitorear el consumo de CPU y memoria RAM.
 - Medir tiempos de respuesta bajo diferentes niveles de carga.
 - Analizar el comportamiento de la base de datos frente a consultas masivas.
 - Evaluar la estabilidad del sistema bajo condiciones de estrés.
-

3. Tecnologías Utilizadas

Durante el desarrollo del proyecto se emplearon las siguientes tecnologías:

Tecnología	Función
Docker	Virtualización y despliegue de servicios
PostgreSQL	Sistema gestor de base de datos
PgAdmin	Administración de la base de datos
Next.js	Desarrollo del dashboard web
React	Construcción de la interfaz de usuario
Recharts	Visualización gráfica de métricas
Ubuntu (WSL)	Entorno de ejecución

4. Metodología

La metodología empleada para el desarrollo del proyecto estuvo compuesta por las siguientes etapas:

Fase 1. Preparación del Entorno

Se instalaron y configuraron Docker, PostgreSQL y PgAdmin dentro de un entorno Ubuntu ejecutado mediante WSL.

Fase 2. Construcción de la Base de Datos

Se diseñaron e implementaron las tablas necesarias para almacenar usuarios, productos y transacciones. Posteriormente se generó una gran cantidad de registros para simular escenarios reales de operación.

Fase 3. Desarrollo del Dashboard

Se construyó una aplicación web utilizando Next.js capaz de consultar información del sistema y mostrar métricas en tiempo real mediante componentes gráficos.

Fase 4. Ejecución de Pruebas

Se realizaron consultas con diferentes volúmenes de registros para evaluar tiempos de respuesta y comportamiento del sistema.

Fase 5. Monitoreo y Análisis

Se utilizaron herramientas como Docker Stats y Htop para observar el consumo de recursos y validar la estabilidad del sistema durante las pruebas.

5. Arquitectura del Sistema

La arquitectura desarrollada está compuesta por tres componentes principales:

1. Base de datos PostgreSQL ejecutándose en un contenedor Docker.
 2. PgAdmin para administración y consulta de datos.
 3. Dashboard web desarrollado con Next.js para monitoreo y ejecución de pruebas.
-

Figura 1. Arquitectura general del sistema y contenedores Docker utilizados.



CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
2ef99690f3eb	dpape/pgadmin4	"/entrypoint.sh"	2 weeks ago	Up 2 hours	0.0.0.0:5050->80/tcp, [::]:5050->80/tcp
1ff6f22e51c5	postgres:16	"docker-entrypoint.s..."	2 weeks ago	Up 2 hours	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp

6. Implementación de la Base de Datos

Se implementó una base de datos relacional compuesta por las tablas:

- usuarios
- productos
- transacciones
- logs_carga

La tabla transacciones fue utilizada como principal fuente de información para las pruebas de rendimiento.

Figura 2. Estructura de tablas de la base de datos.

```
famahu1785@Famahu:~/proyecto-so/stress-dashboard$ docker exec -it postgres-d
b psql -U admin -d appdb
psql (16.14 (Debian 16.14-1.pgdg13+1))
Type "help" for help.

appdb=# SELECT COUNT(*) FROM transacciones;
 count
-----
 520000
(1 row)

appdb=# |
```

Posteriormente se verificó la correcta carga de datos, alcanzando más de 520.000 registros almacenados.

Figura 3. Verificación del total de registros almacenados.

```
famahu1785@Famahu:~/proyecto-so/stress-dashboard$ docker exec -it postgres-d
b psql -U admin -d appdb
psql (16.14 (Debian 16.14-1.pgdg13+1))
Type "help" for help.

appdb=# \dt
      List of relations
Schema |      Name      | Type  | Owner
-----+-----+-----+-----
public | logs_carga     | table | admin
public | productos      | table | admin
public | transacciones  | table | admin
public | usuarios       | table | admin
(4 rows)

appdb=# SELECT COUNT(*) FROM transacciones;
 count
-----
 520000
(1 row)
```

Adicionalmente se inspeccionaron registros individuales para validar la consistencia de la información generada.

Figura 4. Muestra de registros de la tabla transacciones.

appdb=# SELECT * FROM transacciones LIMIT 5;

id	usuario_id	producto_id	cantidad	created_at
1	812	21	7	2026-06-03 13:42:39.52596
2	742	952	6	2026-06-03 13:42:39.527873
3	886	3781	1	2026-06-03 13:42:39.528897
4	20	1916	8	2026-06-03 13:42:39.529818
5	577	1573	1	2026-06-03 13:42:39.530818

(5 rows)

Active Windows
Ve a Configuración para activar Windows.

7. Desarrollo del Dashboard Web

Se desarrolló un dashboard interactivo capaz de mostrar en tiempo real:

- Consumo de CPU.
- Consumo de memoria RAM.
- Nivel de estrés del sistema.
- Estado operativo.
- Historial gráfico de recursos.
- Resultados de pruebas de estrés.
- Tiempo de respuesta de consultas.

La interfaz fue construida utilizando React y Next.js, incorporando gráficos dinámicos mediante la librería Recharts.

Figura 5. Estado inicial del Dashboard.



En el estado inicial se observa un bajo consumo de recursos, indicando que el sistema se encuentra funcionando de manera estable antes de iniciar las pruebas.

8. Pruebas de Estrés

Se realizaron múltiples pruebas incrementando progresivamente el volumen de registros consultados.

Tabla de Resultados

Registros Consultados Tiempo de Respuesta

1.000	0.09 s
5.000	0.12 s
20.000	0.15 s
499.999	1.17 s

Prueba 1 – Consulta de 1.000 registros

Tiempo de respuesta: 0.09 segundos.

Figura 6. Consulta de 1.000 registros.



La consulta fue ejecutada prácticamente de forma instantánea, evidenciando una baja carga para la base de datos.

Prueba 2 – Consulta de 5.000 registros

Tiempo de respuesta: 0.12 segundos.

Figura 7. Consulta de 5.000 registros.



Se observó un incremento mínimo en el tiempo de respuesta respecto a la prueba anterior.

Prueba 3 – Consulta de 20.000 registros

Tiempo de respuesta: 0.15 segundos.

Figura 8. Consulta de 20.000 registros.



La base de datos continuó respondiendo de forma estable sin impactos significativos sobre CPU o memoria.

Prueba 4 – Consulta de 499.999 registros

Tiempo de respuesta: 1.17 segundos.

Figura 9. Consulta de alta carga.



Esta fue la prueba más exigente del proyecto. A pesar de procesar cerca de medio millón de registros, PostgreSQL mantuvo tiempos de respuesta aceptables y estabilidad operacional.

9. Monitoreo de Recursos

Durante las pruebas se emplearon herramientas externas para observar el comportamiento de los recursos del sistema.

Docker Stats

Docker Stats permitió visualizar el consumo de CPU y memoria de cada contenedor.

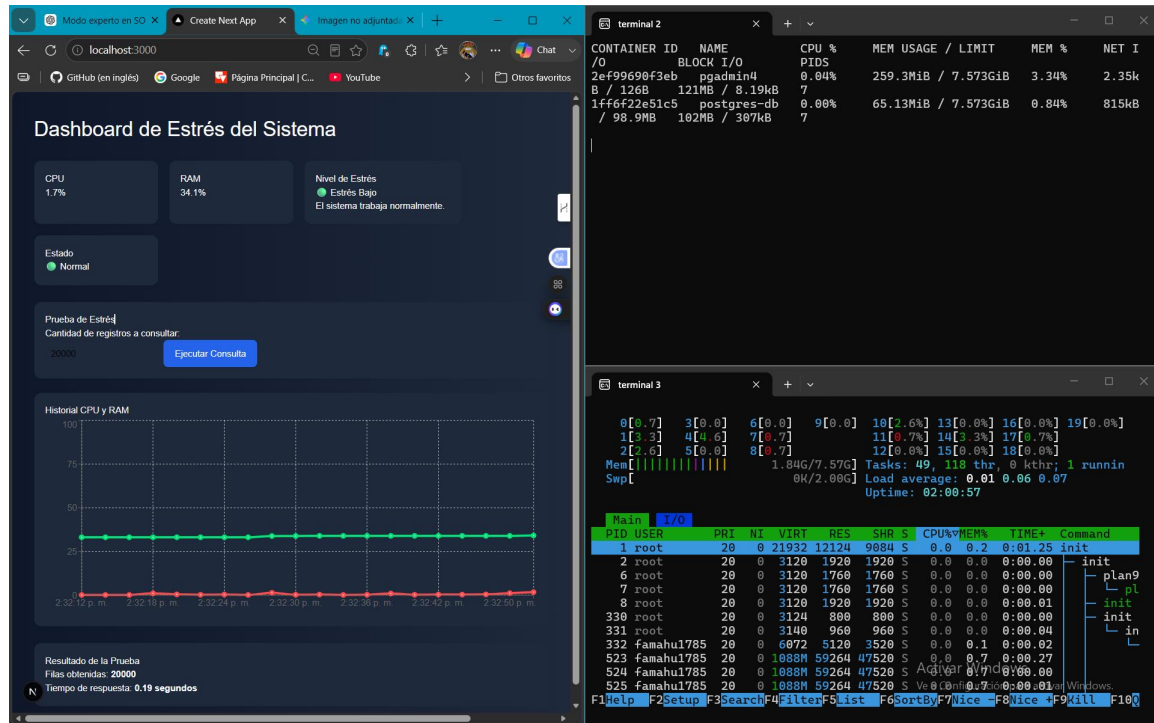
Figura 10. Monitoreo mediante Docker Stats.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
2ef99690f3eb	pgadmin4	0.04%	259.3MiB / 7.573GiB	3.34%	2.35kB / 126B	121MB / 8.19kB	7
1ff6f22e51c5	postgres-db	0.00%	62.18MiB / 7.573GiB	0.80%	816kB / 98.9MB	102MB / 307kB	6

Htop

La herramienta Htop permitió observar procesos activos, carga del procesador, utilización de memoria y estado general del sistema operativo.

Figura 11. Monitoreo mediante Htop.



10. Análisis de Resultados

Los resultados obtenidos muestran una relación directamente proporcional entre el volumen de registros consultados y el tiempo de respuesta de la base de datos. A medida que aumentó la cantidad de datos procesados, los tiempos crecieron de manera gradual y controlada.

Las consultas pequeñas presentaron tiempos inferiores a 0.15 segundos, mientras que la consulta de 499.999 registros alcanzó aproximadamente 1.17 segundos. A pesar del incremento de carga, el sistema no presentó errores, bloqueos ni caídas del servicio.

El monitoreo realizado mediante Docker Stats y Htop evidenció que el consumo de memoria permaneció dentro de límites seguros y que el uso del procesador nunca alcanzó niveles críticos durante las pruebas realizadas.

Los resultados permiten concluir que PostgreSQL ofrece un comportamiento estable y adecuado para manejar grandes volúmenes de información dentro de las condiciones evaluadas.

11. Conclusiones

- Se implementó exitosamente una plataforma de pruebas de estrés utilizando Docker, PostgreSQL y Next.js.
 - El dashboard desarrollado permitió monitorear recursos del sistema en tiempo real.
 - PostgreSQL respondió de manera eficiente incluso al consultar cerca de medio millón de registros.
 - Docker facilitó el aislamiento y administración de los servicios utilizados.
 - El consumo de recursos permaneció dentro de parámetros estables durante todas las pruebas.
 - La solución desarrollada demuestra ser escalable y adecuada para escenarios de carga moderada y alta.
-

12. Referencias

[1] PostgreSQL Global Development Group, "PostgreSQL Documentation". Disponible en: <https://www.postgresql.org/docs/>

[2] Docker Inc., "Docker Documentation". Disponible en: <https://docs.docker.com/>

[3] Vercel, "Next.js Documentation". Disponible en: <https://nextjs.org/docs>

[4] Meta, "React Documentation". Disponible en: <https://react.dev>

[5] pgAdmin Development Team, "pgAdmin Documentation". Disponible en: <https://www.pgadmin.org/docs/>

